

Advanced Data Structures: Mastering the Queue

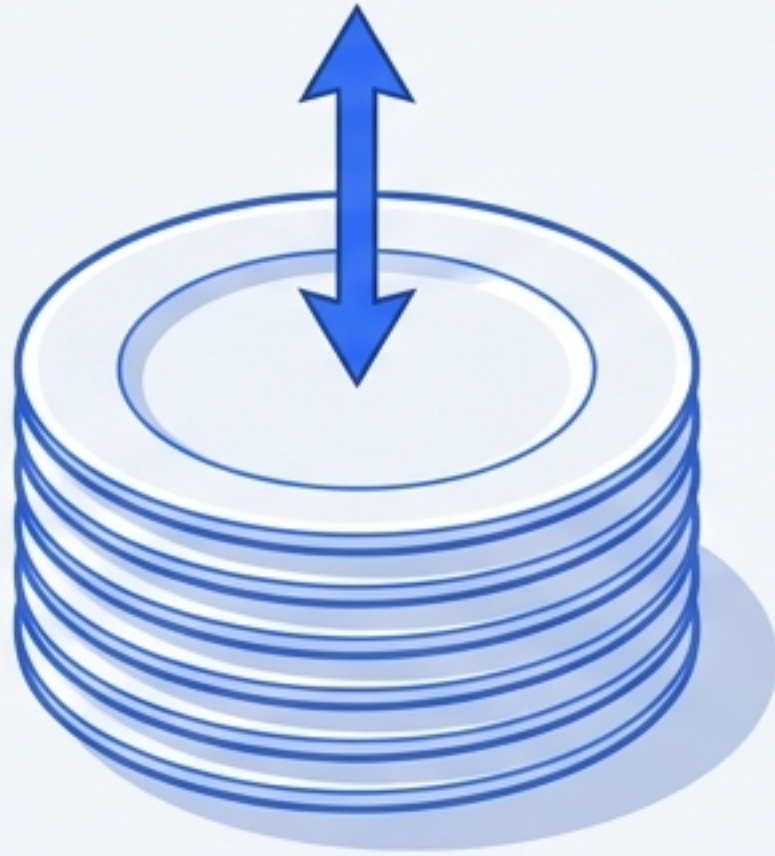
From Conceptual Theory to LeetCode Implementation



Roboto

Based on the iCodeGuru Bootcamp Curriculum | Week 2, Day 4

The Core Mechanism: FIFO vs. LIFO



LIFO (Last In, First Out)

The last plate placed is the first one removed.

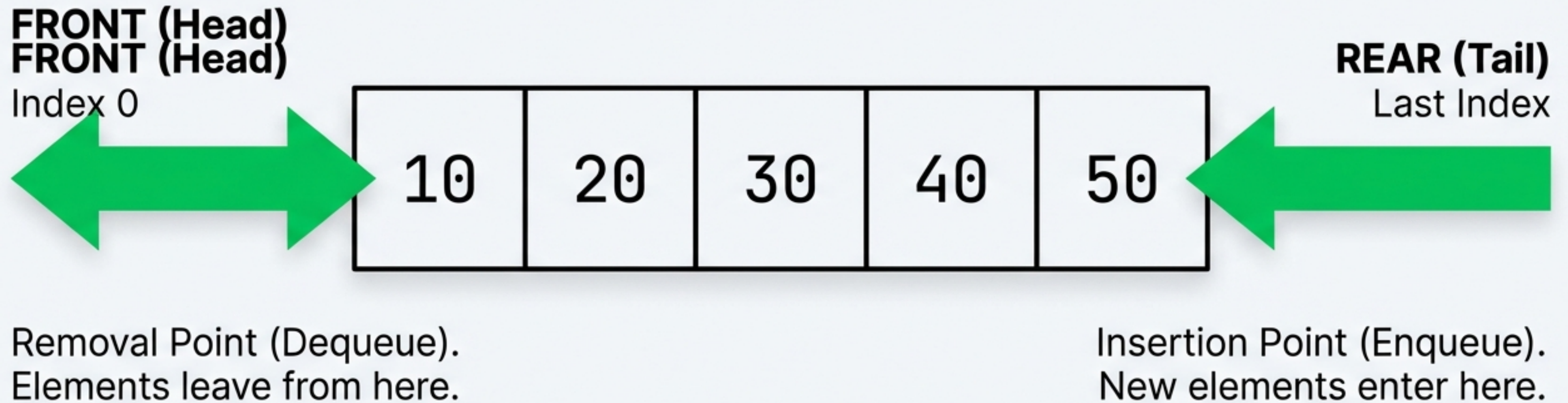


FIFO (First In, First Out)

The first person in line is the first one served.

Key Insight: In a Queue, the element that enters first is the one to leave first. This is the structural opposite of a Stack.

Anatomy of a Queue



We insert at the **Rear (Last Index)**, and we **delete** from the **Front (Index 0)**.

Essential Operations

Enqueue

Add an element to the Rear.



Dequeue

Remove the element from the Front.



Peek / Front

View the first element without removing it.



IsEmpty

Check if the queue has data.



The Printer Queue Analogy



The first document sent is the first to print. The 5th document sent must wait for the previous 4 to finish.

The LeetCode Challenge

Problem: Implement a First In, First Out (FIFO) Queue using only two Stacks.

Allowed Tools (Stacks Only)

- push (to top)
- pop (from top)
- peek (at top)
- size
- empty

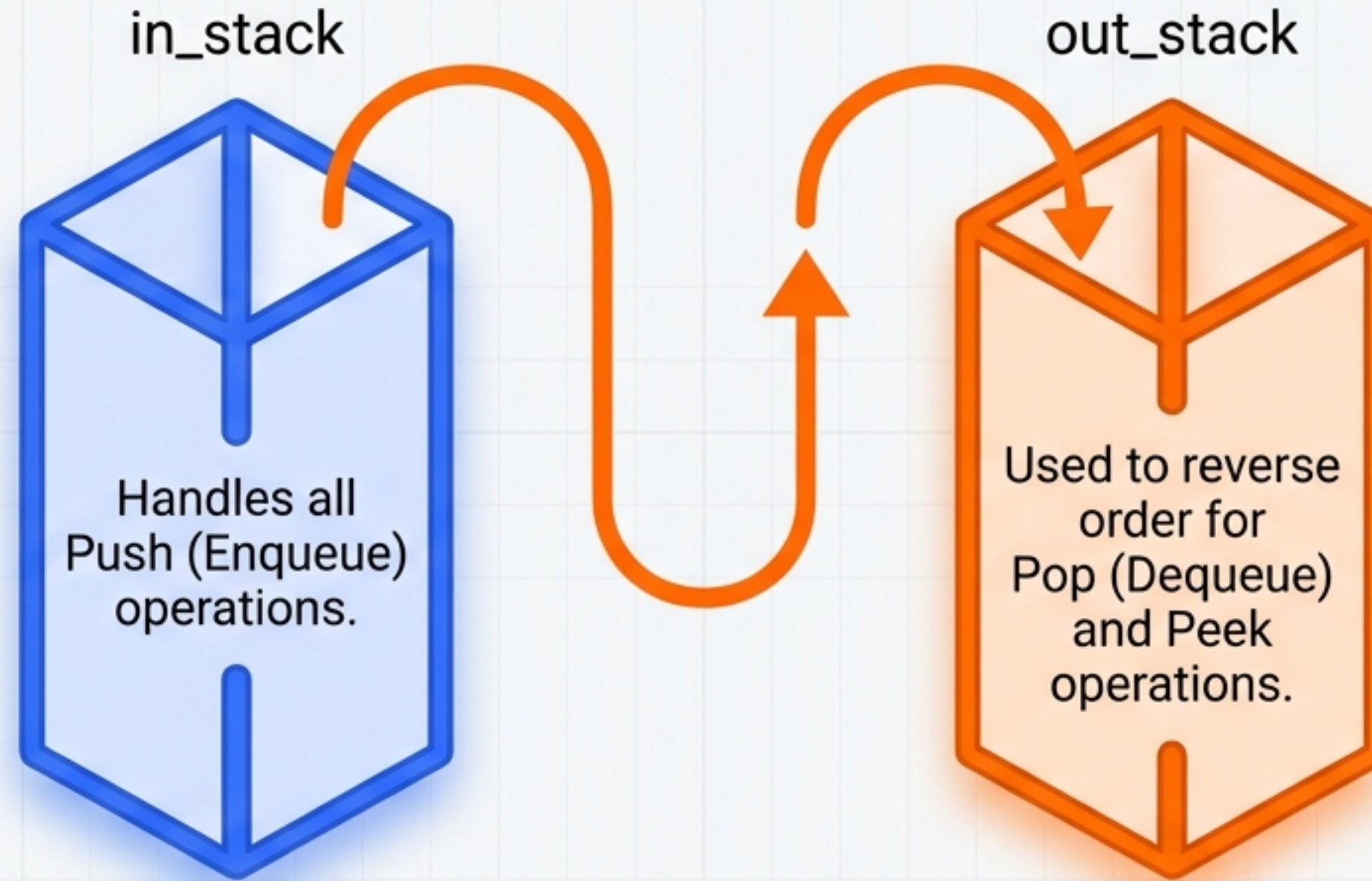


The Conflict

How do we reverse the order to get the OLDEST element out, when Stacks only give us the NEWEST element?



The Strategy: Two Stacks

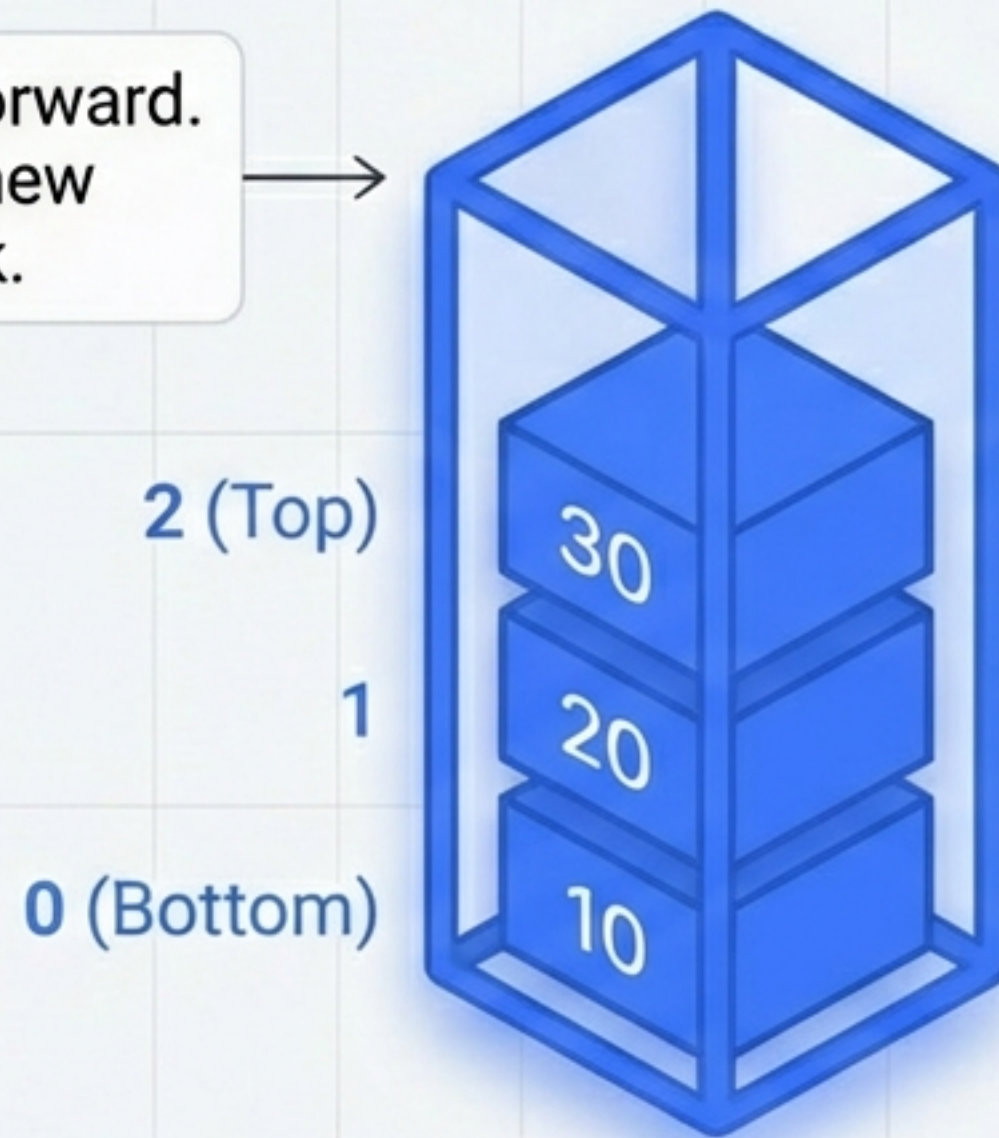


We use the second stack to reverse the order of elements, effectively turning LIFO back into FIFO.

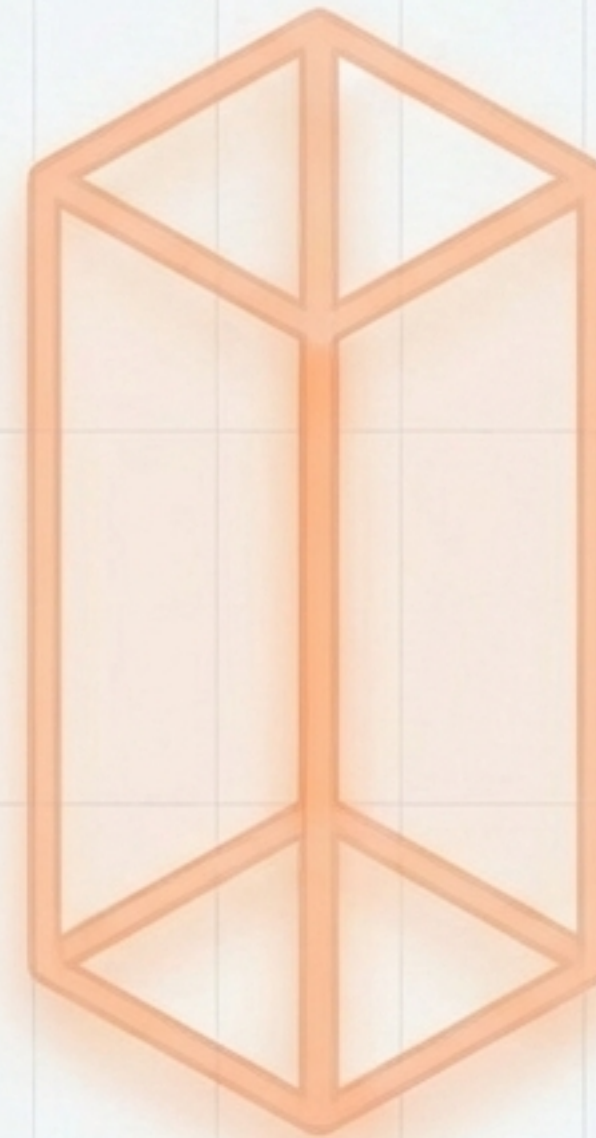
Step 1: The Push Operation (Enqueue)

Action: Push values 10, 20, and 30.

Input is straightforward.
We simply push new
data into in_stack.

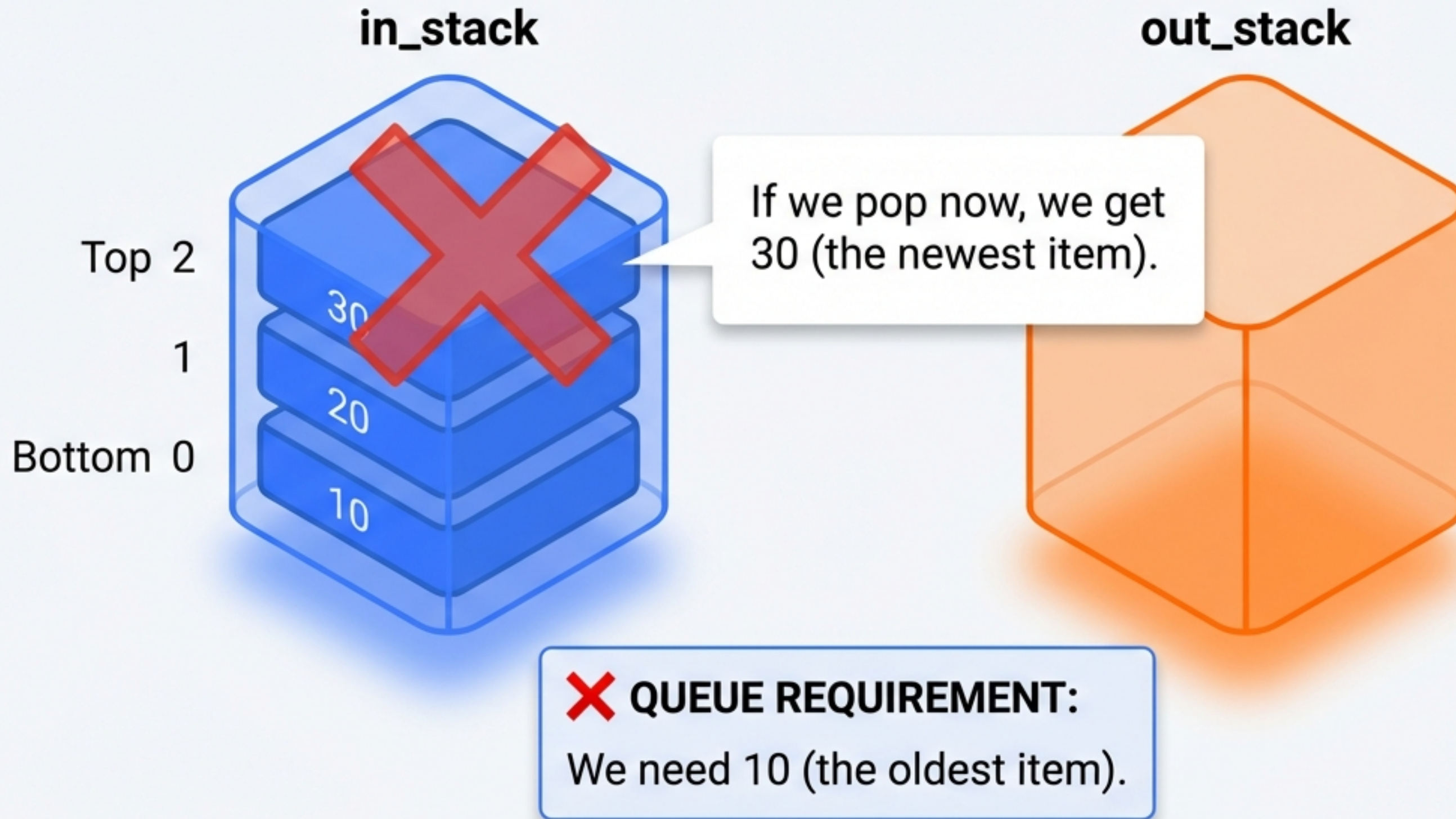


in_stack



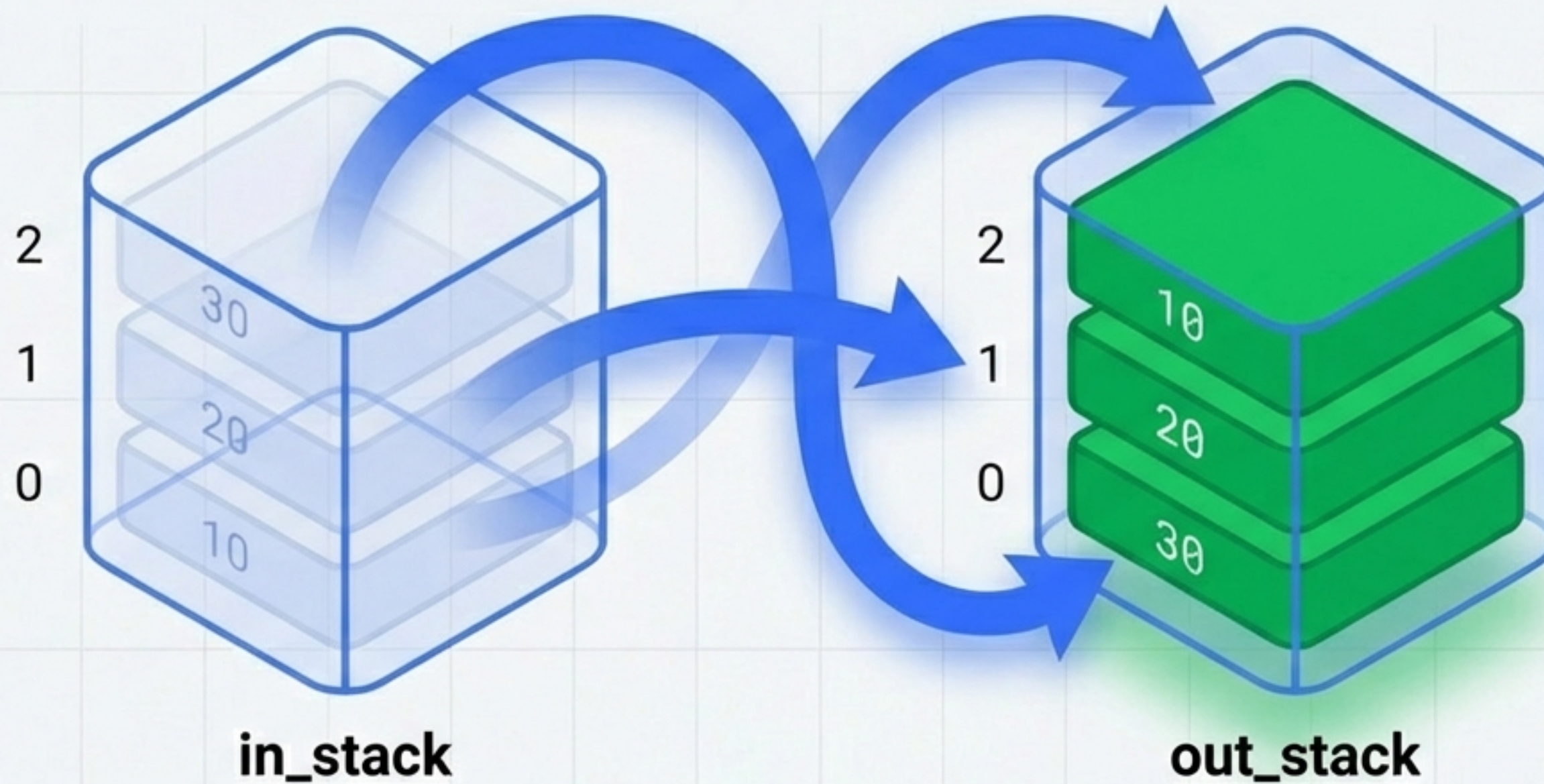
out_stack

The Problem with Direct Pop



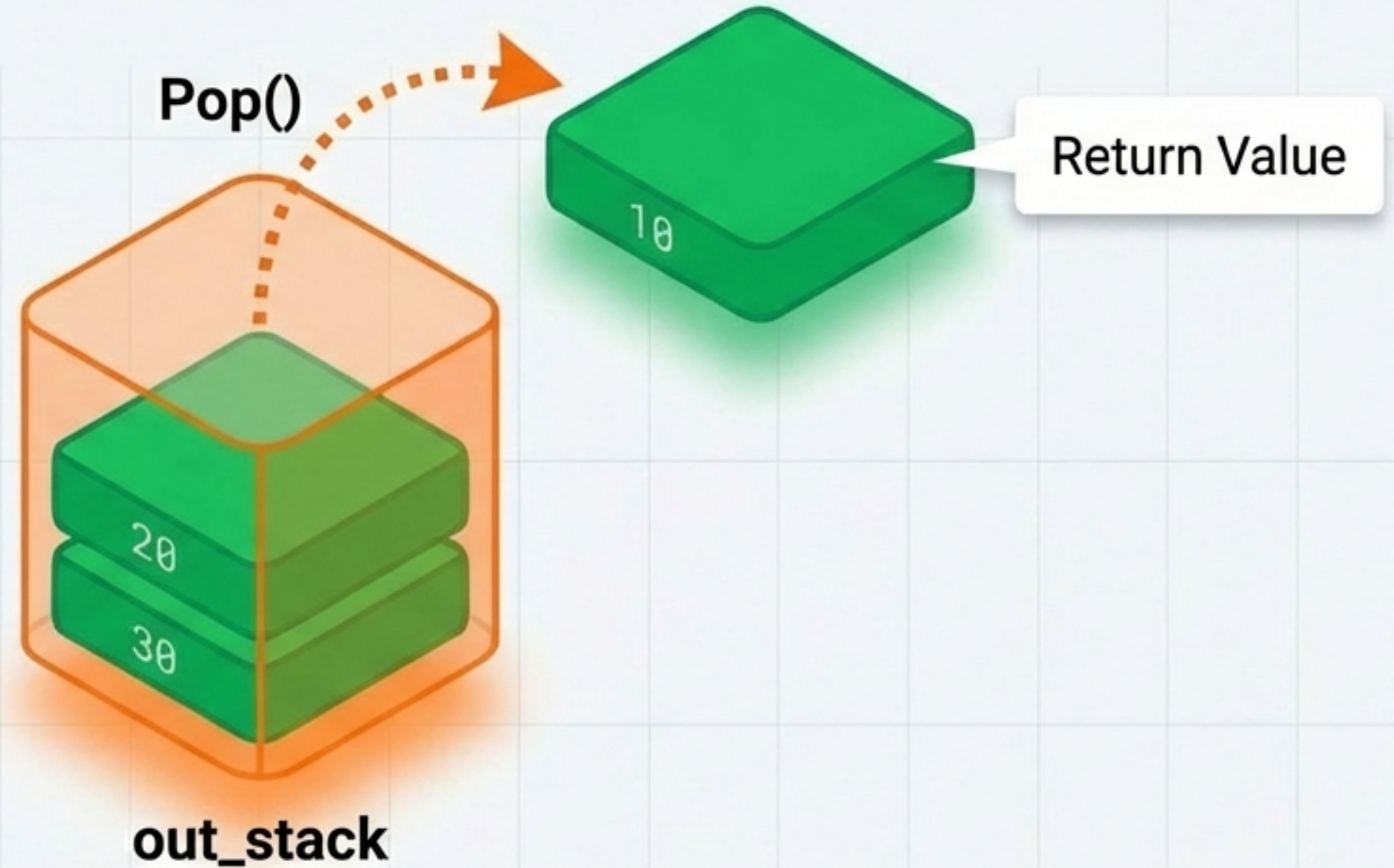
Solution: We must transfer the data to access the bottom element.

Step 2: The Transfer (Reversing Order)



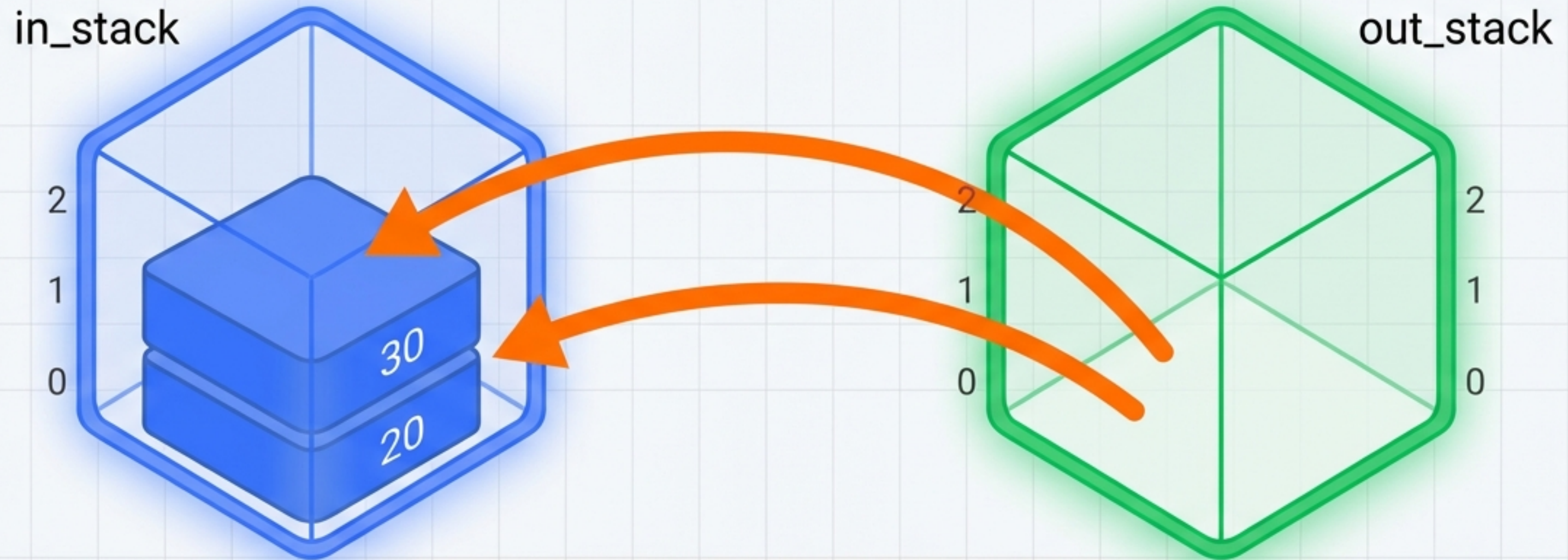
Transformation: By popping from one and pushing to the other, the order is reversed. 10 is now at the top.

Step 3: The Pop Operation (Dequeue)



We pop from out_stack. Since the order was reversed, we retrieve 10. We have successfully achieved FIFO behavior.

Step 4: The Reset (Restoring State)



Transfer remaining elements back to in_stack

1. Pop remaining elements from out_stack.
 2. Push them back into in_stack.
- This prepares in_stack to receive new elements directly on top of the current queue.

Implementation: Class Structure & Push

```
class MyQueue:
    def __init__(self):
        # Initialize two stacks
        self.in_stack = []
        self.out_stack = []

    def push(self, x: int) -> None:
        # Simply append to the input stack
        self.in_stack.append(x)
```

`__init__` creates our two lists. `push` simply appends to the primary stack ($O(1)$).

Inter Tight: Implementation: Pop Logic

Phase 1:
Reverse Order

Phase 2:
Extract

Phase 3:
Reset

```
1
2  def pop(self) -> int:
3      # 1. Move all to out_stack to reverse order
4      while self.in_stack:
5          self.out_stack.append(self.in_stack.pop())
6
7      # 2. Pop the target element (FIFO achieved)
8      val = self.out_stack.pop()
9
10     # 3. Move remaining back to in_stack
11     while self.out_stack:
12         self.in_stack.append(self.out_stack.pop())
13
14     return val
```

Implementation: Peek Logic

```
def peek(self) -> int:
    # Move all to out_stack
    while self.in_stack:
        self.out_stack.append(self.in_stack.pop())

    # View the top element (Index -1)
    val = self.out_stack[-1]

    # Move all back to in_stack
    while self.out_stack:
        self.in_stack.append(self.out_stack.pop())

    return val
```

Difference from Pop:
We use index [-1] to view the element without removing it.

Analysis & Key Takeaways

Time Complexity

$O(N)$

For Pop/Ppeek operations (due to shuttle transfer).

Space Complexity

$O(N)$

Linear space required to store elements.

- ✓ Queues are FIFO; Stacks are LIFO.
- ✓ We used `in_stack` to capture arrival order.
- ✓ We used `out_stack` to reverse that order for processing.
- ✓ This pattern simulates one data structure using another.